VULNERABILITY TESTING / LAB REPORT

CSRF INTRODUCTION

Web Application Penetration Testing – Internship Report

Organization: IqraSec Academy
Platform: TryHackMe

COVER PAGE

Field	Details
Intern Name	Rohit Kushwah
Organization	IqraSec Academy
Internship	Web Application Penetration Testing (WAPT)
Topic	Cross-Site Request Forgery (CSRF)
Platform	TryHackMe
Number of Labs	1
Completion Date	25-10-2026

This report documents the objective, vulnerability concept, testing methodology, evidence, impact, remediation, and learning outcomes of the CSRF Introduction lab.

Lab 1: CSRF Introduction

1. Lab Name

CSRF Introduction – Cross-Site Request Forgery

Platform: TryHackMe
Room URL: https://tryhackme.com/room/csrfintroduction

2. Objective

The objective of this lab is to understand Cross-Site Request Forgery (CSRF), how it can affect web applications that rely on a user's authenticated session, and how developers can prevent unauthorized requests.

The main objectives are:

- Understand the concept of CSRF.
- Learn how browsers handle authenticated requests and session cookies.
- Identify application actions that may be vulnerable to CSRF.
- Understand the role of anti-CSRF tokens.
- Learn about cookie security attributes and origin validation.
- Understand secure implementation and remediation techniques.

3. Vulnerability / Concept

What is CSRF?

Cross-Site Request Forgery (CSRF) is a web security vulnerability in which an attacker tricks an authenticated user's browser into sending an unwanted request to a web application where the user is already logged in.

If the application accepts the request without adequate CSRF protection, an action may be performed using the victim's existing session.

How CSRF Works

A typical CSRF scenario involves the following sequence:

1. A user logs in to a legitimate web application.
2. The browser stores the session cookie.
3. The user visits a malicious or compromised website.
4. That website causes the browser to send a request to the legitimate application.
5. The browser may automatically attach the relevant session cookie.
6. If the application does not properly validate the request, an unauthorized action may occur.

Important: CSRF does not automatically allow an attacker to read the victim's private response or bypass every browser security control. Exploitability depends on the application's behavior, authentication mechanism, cookie settings, and request protections.

Common CSRF Targets

- Changing account details.
- Updating an email address or password.
- Changing account preferences.
- Performing financial or administrative actions.
- Creating or modifying records.

Common CSRF Protection Mechanisms

1. Anti-CSRF Tokens

The server generates a unique, unpredictable token and validates it when a sensitive request is submitted.

2. SameSite Cookies

The SameSite cookie attribute can restrict when cookies are sent with cross-site requests. Its effectiveness depends on the chosen setting and request context.

3. Origin and Referer Validation

The server can validate the request's origin information where appropriate. These checks should be implemented carefully and should not replace other required controls.

4. Reauthentication for Sensitive Actions

High-risk operations may require users to reauthenticate or confirm the action.

5. Appropriate HTTP Methods

State-changing operations should not be implemented using GET requests. However, using POST alone does not prevent CSRF.

4. Testing Approach

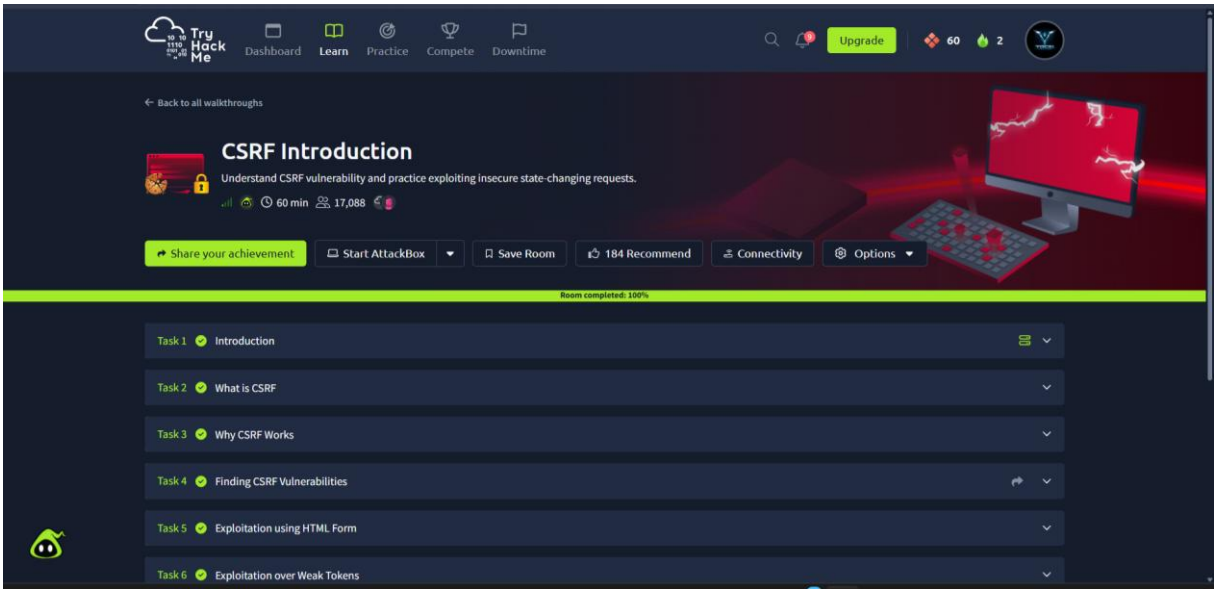
Use the following workflow within the authorized TryHackMe lab environment.

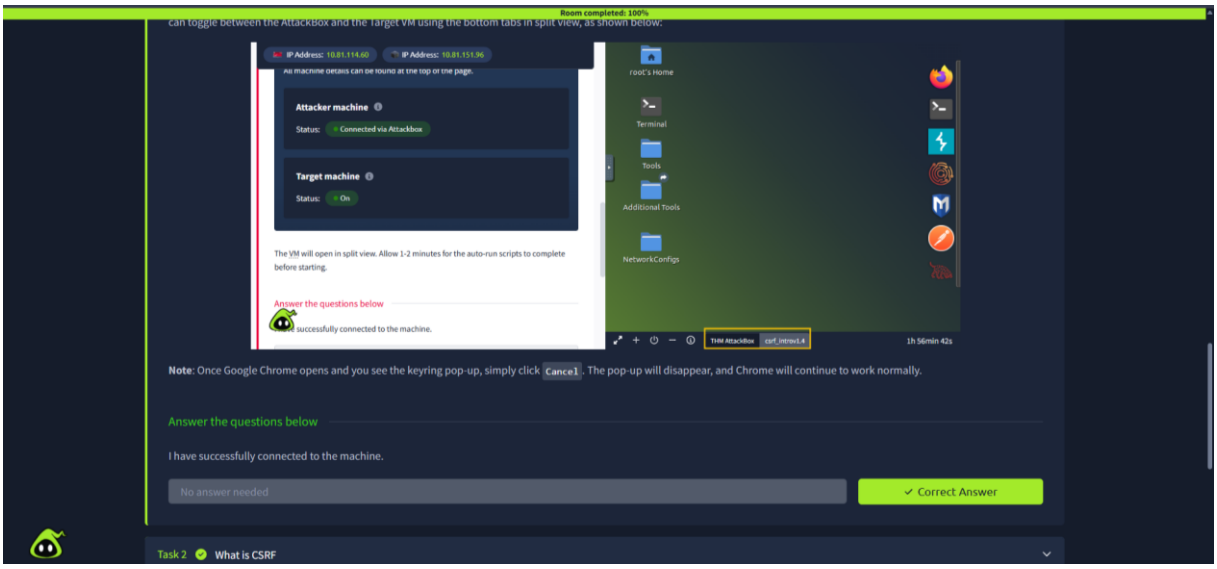
1. Open the CSRF Introduction room on TryHackMe.
2. Read the room instructions and understand the assigned tasks.
3. Start the target machine or lab environment if required.
4. Open the target web application in the provided browser.
5. Identify the authentication process and sign in using the lab-provided account, if required.
6. Identify actions that modify application state, such as updating a profile.
7. Observe the HTTP requests generated when a state-changing action is performed.
8. Record the request method, relevant parameters, and authentication mechanism.
9. Examine whether the application uses an anti-CSRF token.
10. Determine whether the token is validated by the server, following the lab instructions.
11. Observe how the application handles missing or invalid tokens, where the lab permits testing.
12. Review the session cookie attributes, including SameSite, if visible and relevant.
13. Examine whether the application validates the request origin or other relevant request context.
14. Determine whether state-changing actions are incorrectly exposed through GET requests.
15. Follow the lab's instructions to perform its designated CSRF demonstration.
16. Observe the application's response and whether the requested action succeeds.
17. Record the result without accessing accounts or data outside the authorized lab.

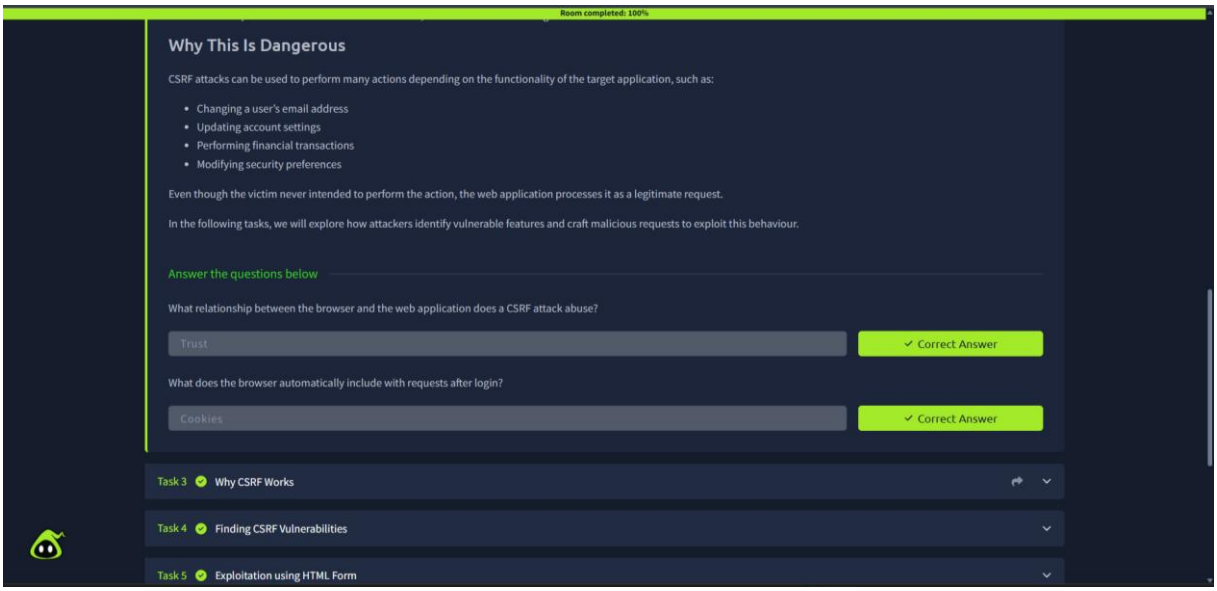
18. Capture screenshots of the relevant request, response, and outcome.
19. Compare the observed behavior with the expected CSRF protection mechanisms.
20. Document the confirmed findings and recommend appropriate remediation.

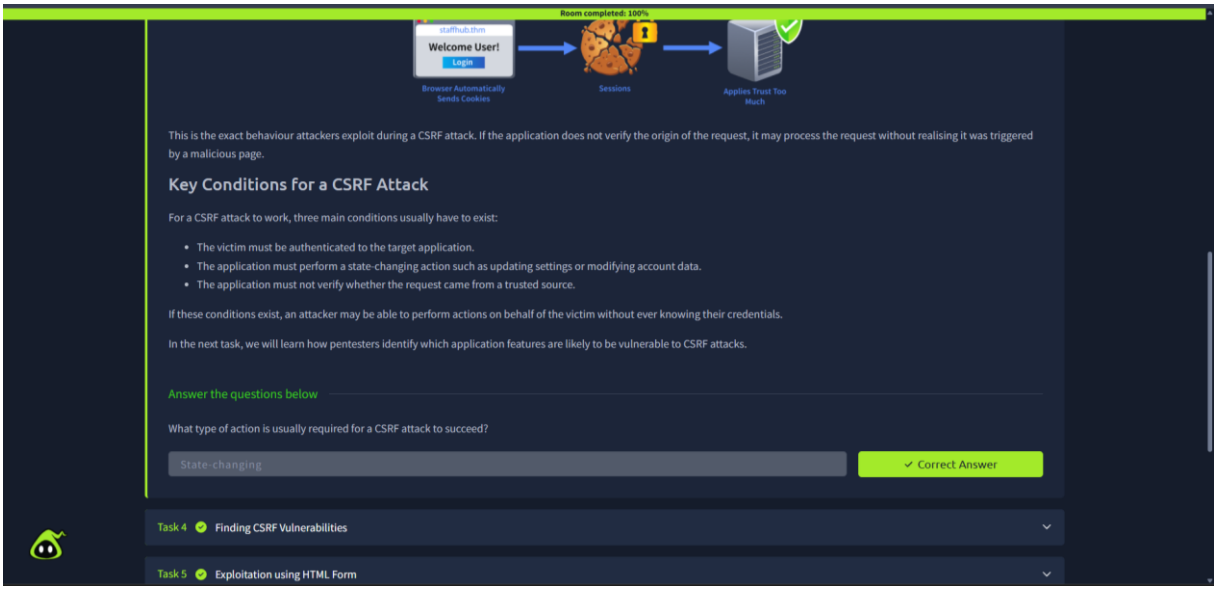
Testing note: Record only the tests actually performed and the outcomes actually observed. Do not claim that a CSRF vulnerability was confirmed merely because the room covers the topic.

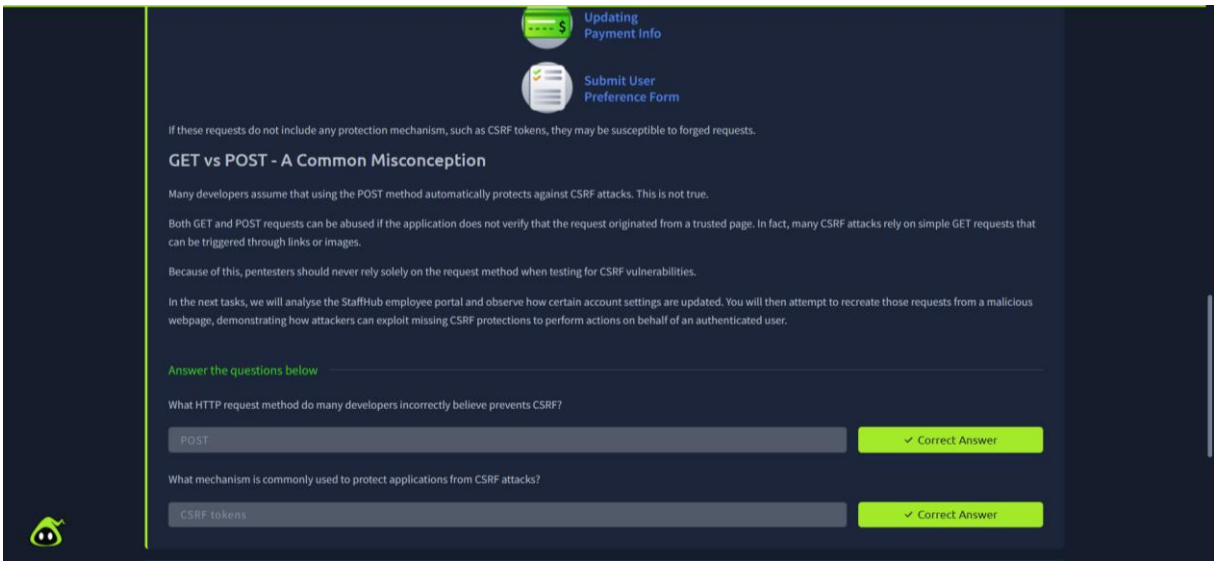
5. Evidence / Screenshots

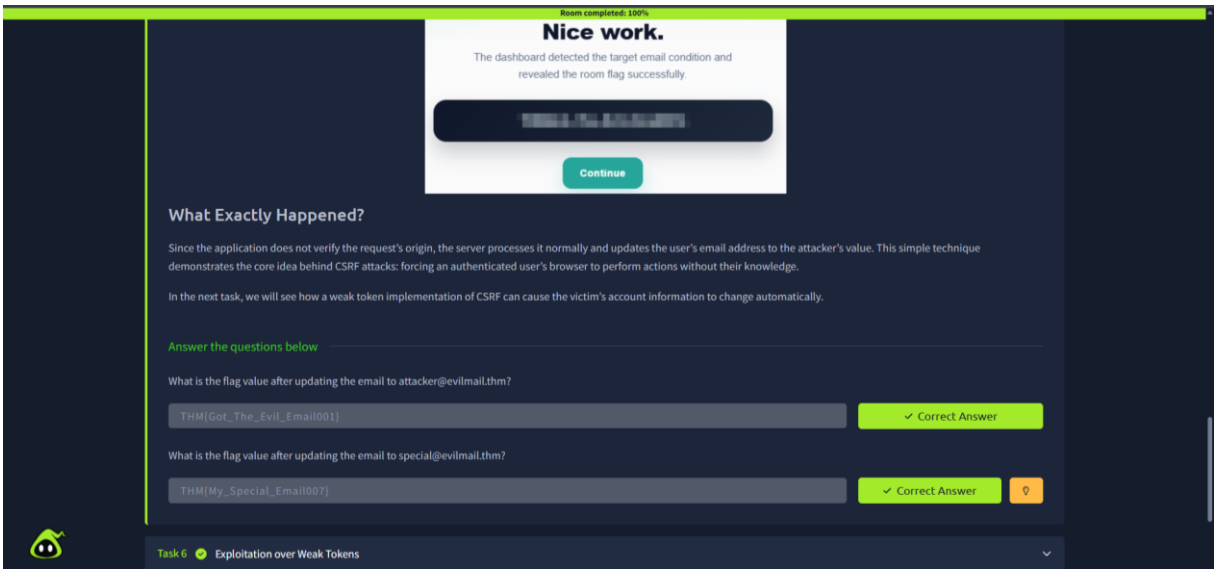


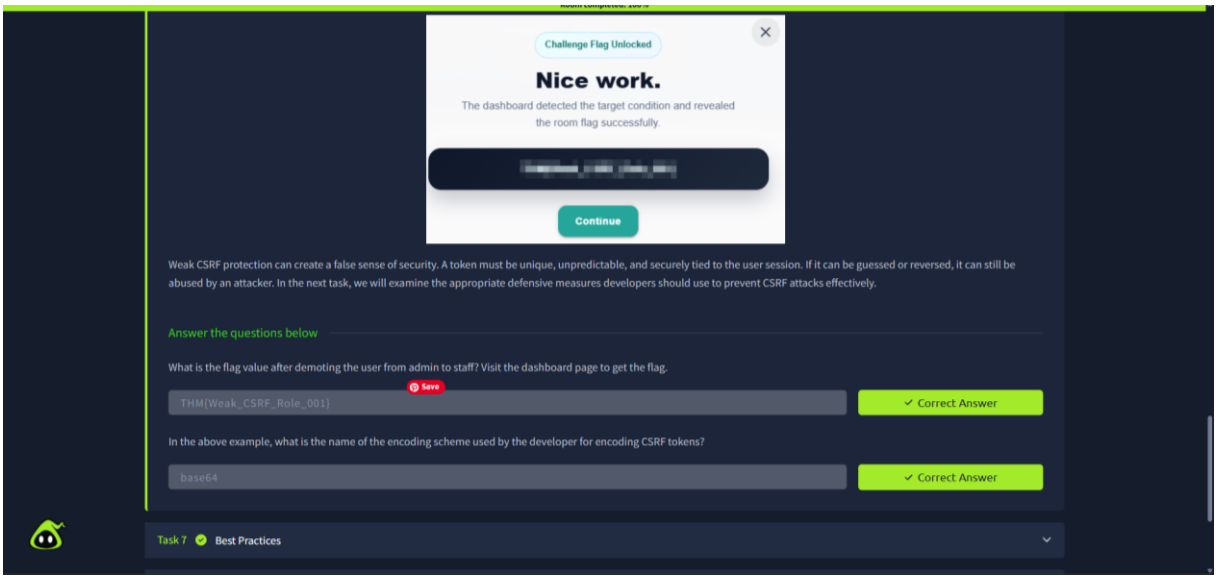


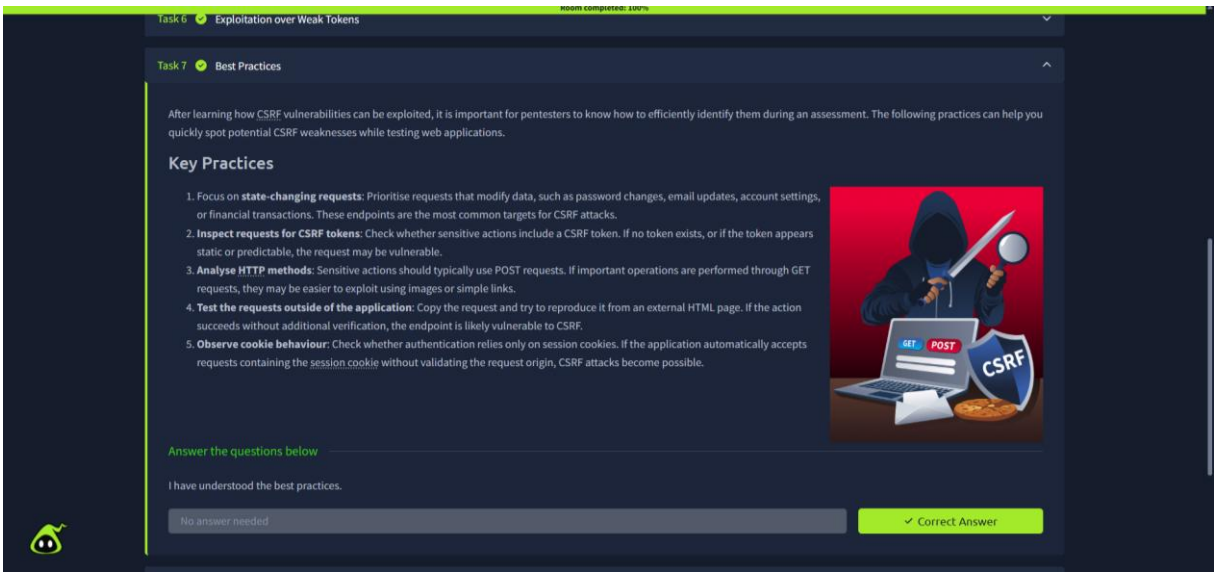

Room completed: 100%
Why This Is Dangerous
CSRF attacks can be used to perform many actions depending on the functionality of the target application, such as:
Changing a user's email address
Updating account settings
Performing financial transactions
Modifying security preferences
Even though the victim never intended to perform the action, the web application processes it as a legitimate request.
In the following tasks, we will explore how attackers identify vulnerable features and craft malicious requests to exploit this behaviour.
Answer the questions below
What relationship between the browser and the web application does a CSRF attack abuse?
Trust
✓ Correct Answer
What does the browser automatically include with requests after login?
Cookies
✓ Correct Answer
Task 3 Why CSRF Works
Task 4 Finding CSRF Vulnerabilities
Task 5 Exploitation using HTML Form

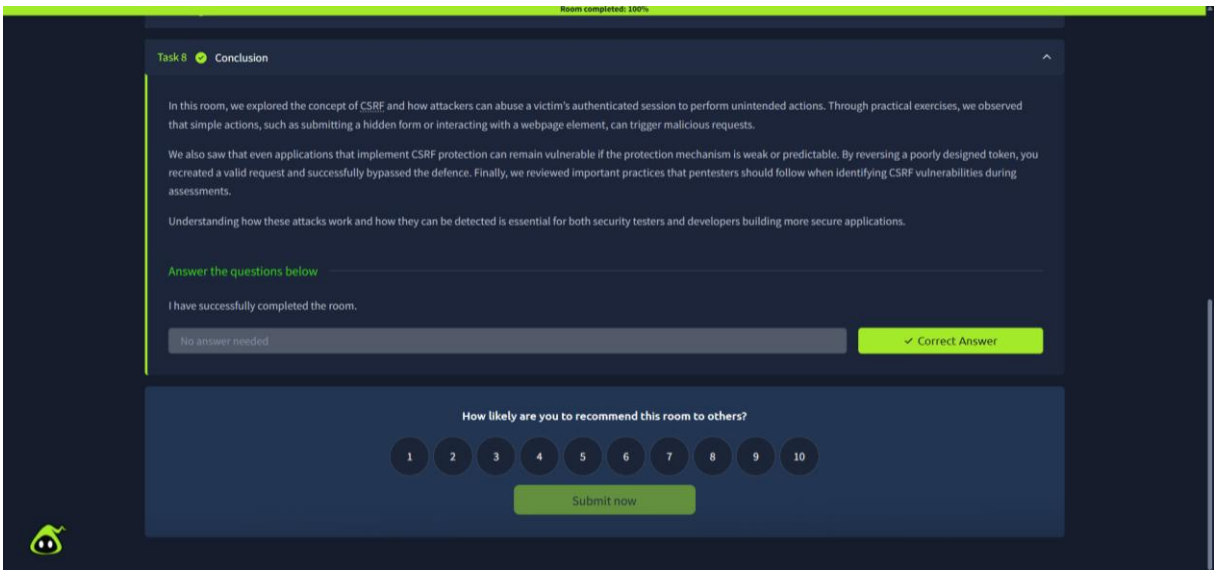

Room completed: 100%
staffhub.thm
Welcome User!
Login
Browser Automatically Sends Cookies
Sessions
Applies Trust Too Much
This is the exact behaviour attackers exploit during a CSRF attack. If the application does not verify the origin of the request, it may process the request without realising it was triggered by a malicious page.
Key Conditions for a CSRF Attack
For a CSRF attack to work, three main conditions usually have to exist:
The victim must be authenticated to the target application.
The application must perform a state-changing action such as updating settings or modifying account data.
The application must not verify whether the request came from a trusted source.
If these conditions exist, an attacker may be able to perform actions on behalf of the victim without ever knowing their credentials.
In the next task, we will learn how pentesters identify which application features are likely to be vulnerable to CSRF attacks.
Answer the questions below
What type of action is usually required for a CSRF attack to succeed?
State-changing
✓ Correct Answer
Task 4 Finding CSRF Vulnerabilities
Task 5 Exploitation using HTML Form


Updating
Payment Info
Submit User
Preference Form
If these requests do not include any protection mechanism, such as CSRF tokens, they may be susceptible to forged requests.
GET vs POST - A Common Misconception
Many developers assume that using the POST method automatically protects against CSRF attacks. This is not true.
Both GET and POST requests can be abused if the application does not verify that the request originated from a trusted page. In fact, many CSRF attacks rely on simple GET requests that can be triggered through links or images.
Because of this, pentesters should never rely solely on the request method when testing for CSRF vulnerabilities.
In the next tasks, we will analyse the StaffHub employee portal and observe how certain account settings are updated. You will then attempt to recreate those requests from a malicious webpage, demonstrating how attackers can exploit missing CSRF protections to perform actions on behalf of an authenticated user.
Answer the questions below
What HTTP request method do many developers incorrectly believe prevents CSRF?
POST
✓ Correct Answer
What mechanism is commonly used to protect applications from CSRF attacks?
CSRF tokens
✓ Correct Answer


Room completed: 100%
Nice work.
The dashboard detected the target email condition and revealed the room flag successfully.
Continue
What Exactly Happened?
Since the application does not verify the request's origin, the server processes it normally and updates the user's email address to the attacker's value. This simple technique demonstrates the core idea behind CSRF attacks: forcing an authenticated user's browser to perform actions without their knowledge.
In the next task, we will see how a weak token implementation of CSRF can cause the victim's account information to change automatically.
Answer the questions below
What is the flag value after updating the email to attacker@evilmail.thm?
THM{Got_The_Evil_Email001}
✓ Correct Answer
What is the flag value after updating the email to special@evilmail.thm?
THM{My_Special_Email007}
✓ Correct Answer
Task 6
Exploitation over Weak Tokens


Challenge Flag Unlocked
Nice work.
The dashboard detected the target condition and revealed the room flag successfully.
Continue
Weak CSRF protection can create a false sense of security. A token must be unique, unpredictable, and securely tied to the user session. If it can be guessed or reversed, it can still be abused by an attacker. In the next task, we will examine the appropriate defensive measures developers should use to prevent CSRF attacks effectively.
Answer the questions below
What is the flag value after demoting the user from admin to staff? Visit the dashboard page to get the flag.
THM{Weak_CSRF_Role_001}
Save
✓ Correct Answer
In the above example, what is the name of the encoding scheme used by the developer for encoding CSRF tokens?
base64
✓ Correct Answer
Task 7 Best Practices


Task 6 Exploitation over Weak Tokens
Task 7 Best Practices
After learning how CSRF vulnerabilities can be exploited, it is important for pentesters to know how to efficiently identify them during an assessment. The following practices can help you quickly spot potential CSRF weaknesses while testing web applications.
Key Practices
1. Focus on state-changing requests: Prioritise requests that modify data, such as password changes, email updates, account settings, or financial transactions. These endpoints are the most common targets for CSRF attacks.
2. Inspect requests for CSRF tokens: Check whether sensitive actions include a CSRF token. If no token exists, or if the token appears static or predictable, the request may be vulnerable.
3. Analyse HTTP methods: Sensitive actions should typically use POST requests. If important operations are performed through GET requests, they may be easier to exploit using images or simple links.
4. Test the requests outside of the application: Copy the request and try to reproduce it from an external HTML page. If the action succeeds without additional verification, the endpoint is likely vulnerable to CSRF.
5. Observe cookie behaviour: Check whether authentication relies only on session cookies. If the application automatically accepts requests containing the session cookie without validating the request origin, CSRF attacks become possible.
GET
POST
CSRF
Answer the questions below
I have understood the best practices.
No answer needed
✓ Correct Answer


Room completed: 100%
Task 8 Conclusion
In this room, we explored the concept of CSRF and how attackers can abuse a victim's authenticated session to perform unintended actions. Through practical exercises, we observed that simple actions, such as submitting a hidden form or interacting with a webpage element, can trigger malicious requests.
We also saw that even applications that implement CSRF protection can remain vulnerable if the protection mechanism is weak or predictable. By reversing a poorly designed token, you recreated a valid request and successfully bypassed the defence. Finally, we reviewed important practices that pentesters should follow when identifying CSRF vulnerabilities during assessments.
Understanding how these attacks work and how they can be detected is essential for both security testers and developers building more secure applications.
Answer the questions below
I have successfully completed the room.
No answer needed
✓ Correct Answer
How likely are you to recommend this room to others?
1 2 3 4 5 6 7 8 9 10
Submit now

6. Impact

If a CSRF vulnerability is exploitable in a real application, it may have the following consequences:

- **Unauthorized actions:** An authenticated user may unknowingly trigger an application action.
- **Account setting changes:** Vulnerable functionality may allow changes to profile information or account settings.
- **Financial impact:** Applications with vulnerable transaction functionality may be exposed to unauthorized operations.
- **Administrative impact:** If a privileged user is affected, unauthorized actions may have a wider impact.
- **Business risks:** Successful exploitation can cause operational disruption, data integrity issues, and reputational damage.

The severity depends on the affected functionality, the user's privileges, the application's CSRF defenses, and the exploitability of the request.

7. Remediation

The following measures help prevent CSRF vulnerabilities.

1. Implement anti-CSRF tokens

Generate unpredictable tokens and validate them server-side for relevant state-changing requests.

2. Use appropriate SameSite cookie settings

Configure session cookies with an appropriate SameSite policy, commonly Lax or Strict where compatible with application requirements. Understand that cookie settings alone may not protect every workflow.

3. Validate request origins

For sensitive operations, validate Origin and, where appropriate, Referer headers as part of a broader defense strategy.

4. Avoid state-changing GET requests

Use appropriate methods for state-changing operations and ensure that POST requests are also protected against CSRF.

5. Protect sensitive actions

Require additional confirmation or reauthentication for high-risk actions when appropriate.

6. Do not rely on CORS as the sole defense

CORS primarily governs cross-origin access to responses. It is not a substitute for proper CSRF protections.

7. Apply secure session management

Use secure session handling and configure cookies with appropriate Secure, HttpOnly, and SameSite attributes.

8. Validate requests on the server

Do not rely solely on client-side JavaScript to enforce security requirements.

9. Test all state-changing functionality

Include profile updates, account changes, administrative actions, and other sensitive workflows in security testing.

10. Perform regression testing

After implementing protections, verify that unauthorized cross-site requests are rejected and legitimate application workflows continue to function.

8. What You Learned

The main learning outcomes of this lab are:

- Understanding the definition and purpose of CSRF.
- Learning how authenticated browser requests can be abused.
- Understanding the role of session cookies in CSRF scenarios.
- Recognizing state-changing application functionality that requires protection.
- Understanding anti-CSRF tokens and server-side token validation.
- Learning how SameSite cookie settings contribute to CSRF defense.
- Understanding why POST requests alone do not prevent CSRF.
- Learning to document requests, responses, evidence, and remediation recommendations.
- Understanding the importance of testing security controls in an authorized environment.

Update these learning outcomes according to the techniques and tasks you actually completed in the room.

Conclusion

Cross-Site Request Forgery is a web application security vulnerability that can allow unauthorized actions to be initiated through an authenticated user's browser when adequate protections are missing.

This lab provides an opportunity to examine state-changing requests, session handling, anti-CSRF tokens, and other defensive mechanisms. Effective prevention requires server-side request validation, appropriate cookie settings, secure session management, and thorough testing of sensitive functionality.

The final report should reflect the actual lab observations, supporting screenshots, and completion status.

Report Prepared By: Rohit Kushwah
Organization: IqraSec Academy
Internship: Web Application Penetration Testing (WAPT)